ΑΝΑΠΤΥΞΗ ΛΟΓΙΣΜΙΚΟΥ ΓΙΑ ΠΛΗΡΟΦΟΡΙΑΚΑ ΣΥΣΤΗΜΑΤΑ

Αναζήτηση k εγγύτερων γειτόνων σε γράφους

Αραστσιάρης Κωνσταντίνος - 1115202100006

Τσιρχόγλου Φίλιππος - 1115202100197

Github Repository: https://github.com/FilipposTsirchoglou/Project-2024.git

Εισαγωγή **3**
Αρχική Υλοποίηση **3**
Γράφος 3
Εύρεση Ελάχιστης Απόστασης 4
Κατασκευή Γράφων με λιγότερα από R σημεία 4
Χρήση dummy nodes 4
Βελτιστοποιήσεις **5**
1. Εύρεση τυχαίου medoid για το vamana index 5
2. Εύρεση ακριβούς medoid για τον filtered vamana index 5
3. Αρχικοποίηση γράφου με τυχαίες ακμές 5
4. Αποθήκευση αποστάσεων σε 1-d matrix για την αποφυγή επανυπολογισμού 6
5. Χρήση κοντινότερων γειτόνων κάθε φίλτρου ως medoid για unfiltered queries 6
6. Χρήση multi-threading 6
Ανάλυση Βελτιστοποιήσεων **7**
Εύρεση τυχαίου medoid για το vamana index 8
Εύρεση ακριβούς medoid για τον filtered vamana index 9
Αρχικοποίηση γράφου με τυχαίες ακμές 11
Αποθήκευση αποστάσεων σε 1-d matrix για την αποφυγή επανυπολογισμού 12
Χρήση κοντινότερων γειτόνων κάθε φίλτρου ως medoid για unfiltered queries 15
Χρήση multi-threading 16
Εκτελέσεις με διαφορετικές παραμέτρους **18**
Συμπεράσματα **19**

Εισαγωγή

Σκοπός της εργασίας αυτής είναι η υλοποίηση εφαρμογής για την προσεγγιστική αναζήτηση k εγγύτερων γειτόνων σε γράφους. Η εργασία αναφέρεται στην κατασκευή KNN γράφων με την χρήση του αλγορίθμου Vamana και την προσεγγιστική αναζήτηση k εγγύτερων γειτόνων πάνω σε αυτούς με την χρήση ή μη φίλτρων.

Στόχος αυτής της αναφοράς είναι η σύντομη επεξήγηση του συνόλου της εφαρμογής, η παρουσίαση βελτιστοποιήσεων στην αρχική υλοποίηση, καθώς και η μέτρηση αλλά και μελέτη των αποτελεσμάτων, από τα οποία θα προκύψει το τελικό συμπέρασμα για την απόδοση της εφαρμογής.

Θα αναφέρουμε και θα αιτιολογήσουμε τις σχεδιαστικές μας επιλογές τόσο στην αρχική υλοποίηση, όσο και σε αυτήν κατά τις βελτιστοποιήσεις, ενώ για κάθε μία από αυτές θα δίνουμε και μια αρχική εκτίμηση για το αποτέλεσμα της. Έπειτα θα ακολουθήσει η λεπτομερής ανάλυση κάθε βελτίωσης και τα αντίστοιχα συμπεράσματα και αποτελέσματα, με την χρήση γραφικών παραστάσεων και σχεδιαγραμμάτων.

Τέλος, θα καταλήξουμε σε μια υλοποίηση με συνδυασμό των παραπάνω βελτιώσεων, ώστε να προκύψει η βέλτιστη εκδοχή της εφαρμογής, η οποία θα εκτελεστεί και θα μελετηθεί για διαφορετικές παραμέτρους.

Αρχική Υλοποίηση

Η αρχική μας υλοποίηση τόσο για τον απλό Vamana όσο και για τον Filtered και stitched Vamana βασίστηκε στις παρακάτω σχεδιαστικές επιλογές :

Γράφος

Η υλοποίηση που επιλέξαμε για την αναπαράσταση του γράφου (Vamana , Filtered Vamana) είναι βασισμένη σε λίστες γειτνίασης. Πιο συγκεκριμένα, δημιουργήθηκε ένα struct graph_node το οποίο αναπαριστά κάθε κόμβο του γράφου. Το κάθε graph_node αποθηκεύει χρήσιμες πληροφορίες για το κάθε κόμβο του γράφου: το id του κόμβου, τα cords του κόμβου, λίστα outgoing με pointers προς τις εξερχόμενες ακμές του κόμβου, vector filters με τα φίλτρα του κάθε κόμβου, πεδίο eucl_dist στο οποίο αποθηκεύεται τοπικά στον κάθε κόμβο η απόστασή του από το query node που μας ενδιαφέρει κάθε φορά. Ο Γράφος, συνολικά , αναπαριστάται από ένα vector που αποθηκεύει struct graph_node, ένα για κάθε point του dataset. Επιλέχθηκε vector καθώς χρειαζόμασταν μια δυναμική δομή που να μπορεί να ανταπεξέλθει και σε μεγαλύτερα dataset, αλλά ταυτόχρονα να υποστηρίζει random access.

Μέσω των παραπάνω επιλογών ο Γράφος δημιουργείται μόνο μια φορά στην μνήμη και μέσω pointers και pass by reference έχουν πρόσβαση σε αυτόν όλες οι συναρτήσεις μειώνοντας έτσι την απαιτούμενη μνήμη.

Εύρεση Ελάχιστης Απόστασης

Για τον υπολογισμό και την εύρεση της ελάχιστης απόστασης ενός κόμβου από κάποιο σημείο ερωτήματος, χρησιμοποιήθηκε η συνάρτηση find_min_eucl, η οποία έχει υλοποιηθεί με την χρήση priority queue. Αυτό προσφέρει άμεση ταξινόμηση κατά την εισαγωγή των αποστάσεων, με την μικρότερη απόσταση να βρίσκεται πάντα στην κορυφή του queue. Έτσι επιτυγχάνεται μεγαλύτερη ταχύτητα, αφού η ταξινόμηση γίνεται με insertion sort μέσω του priority queue που υλοποιείται με min_heap και έχει πολυπλοκότητα O(logn).

Κατασκευή Γράφων με λιγότερα από R σημεία

Κατά την δημιουργία των stitched vamana γράφων , παρατηρήθηκε ότι υπάρχουν φίλτρα, με λιγότερα από R σημεία, γεγονός που εμποδίζει την σωστή δημιουργία του γράφου. Έγινε η παραδοχή, ότι στην περίπτωση αυτή, θα ενώνεται το κάθε σημείο με όλα τα υπόλοιπα ίδιου φίλτρου, ώστε να μην επηρεάζεται η r-regular ιδιότητα του γράφου.

Χρήση dummy nodes

Επειδή κάποια φίλτρα έχουν λιγότερα από k σε πλήθος σημεία, δεν υφίσταται η εύρεση των k εγγύτερων γειτόνων. Για τον λόγο αυτό, στην λίστα με τους k εγγύτερους γείτονες που επιστρέφει η greedy search, προστίθενται dummy nodes για τον σωστό υπολογισμό του recall.

Βελτιστοποιήσεις

Προκειμένου να βελτιωθεί η αρχική υλοποίηση της εφαρμογής τόσο σε χρόνο όσο και σε ακρίβεια αποτελεσμάτων (recall), καταλήξαμε στις παρακάτω ενδεχόμενες βελτιώσεις:

1. **Εύρεση τυχαίου medoid για το vamana index**

   Ο υπολογισμός του ακριβούς medoid για τον vamana index είναι χρονοβόρος. Έγινε η δοκιμή τυχαίας επιλογής medoid απο το σύνολο των σημείων.

   Αναμενόμενο αποτέλεσμα: Βελτίωση στον χρόνο κατασκευής του index και του προγράμματος συνολικά και μείωση στην ακρίβεια των αποτελεσμάτων (recall).

2. **Εύρεση ακριβούς medoid για τον filtered vamana index**

   Ο υπολογισμός του medoid για τον filtered vamana index γίνεται με τυχαίο τρόπο για την περίπτωση που το κάθε σημείο έχει μόνο ένα φίλτρο. Έγινε η δοκιμή του ακριβούς medoid απο το σύνολο των σημείων.

Αναμενόμενο αποτέλεσμα: Βελτίωση στην ακρίβεια των αποτελεσμάτων αλλά σαφής αύξηση στον χρόνο εκτέλεσης.

3. Αρχικοποίηση γράφου με τυχαίες ακμές

Κατά την δημιουργία του filtered vamana index ο γράφος αρχικά είναι κενός και κατα το indexing phase προστίθενται σε αυτόν ακμές μεταξυ σημείων με ίδιο φίλτρο. Έγινε δοκιμή αρχικοποίησης του γράφου με τυχαίες ακμές μεταξύ ίδιων φίλτρων και μη.

Αναμενόμενο αποτέλεσμα: Αύξηση στην ακρίβεια αποτελεσμάτων για τα unfiltered queries και αύξηση στον χρόνο εκτέλεσης.

4. Αποθήκευση αποστάσεων σε 1-d matrix για την αποφυγή επανυπολογισμού

Στην αρχική υλοποίηση παρατηρήθηκε περιττός επανυπολογισμός αποστάσεων από κάποιο σημείο προς ένα άλλο μέσα στην ίδια κλήση συνάρτησης. Έγινε δοκιμή αποθήκευσης των ήδη υπολογισμένων αποστάσεων.

Αναμενόμενο αποτέλεσμα: Σαφής βελτίωση στον χρόνο εκτέλεσης.

5. Χρήση κοντινότερων γειτόνων κάθε φίλτρου ως medoid για unfiltered queries

Στην αρχική υλοποίηση παρατηρήθηκε πολύ χαμηλή ακρίβεια αποτελεσμάτων για unfiltered queries στον filtered vamana index, γεγονός που δικαιολογείται από την τυχαία επιλογή medoid για κάθε φίλτρο και από το prioritization του γράφου στα φίλτρα. Έγινε δοκιμή επιλογής κοντινότερου γείτονα για κάθε φίλτρο, ως medoid και αύξηση του L, για extended search, ώστε να ληφθούν υπόψη σημεία που παρά το διαφορετικό τους φίλτρο, έχουν μικρότερη απόσταση.

* Η παραπάνω βελτίωση είναι βασισμένη στην ανάλυση συναδέλφου στο Piazza: https://piazza.com/class/m1kh0ggogpyg0/post/70

Αναμενόμενο αποτέλεσμα: Σαφής αύξηση στον χρόνο εκτέλεσης, αλλά σημαντική βελτίωση στην ακρίβεια των αποτελεσμάτων.

6. Χρήση multi-threading

Τελευταία βελτίωση στην εφαρμογή είναι η χρήση multi-threading, ώστε να παραλληλοποιηθούν κομμάτια του κώδικα και να μειωθεί ο χρόνος εκτέλεσης. Το multi-threading έχει εφαρμοστεί στην τελική μορφή της εφαρμογής, ώστε να την κάνει ακόμη πιο αποδοτική

Αναμενόμενο αποτέλεσμα: Σαφής μείωση στον χρόνο εκτέλεσης.

Ανάλυση Βελτιστοποιήσεων

Στον πίνακα που ακολουθεί φαινονται οι χρόνοι εκτέλεσης και τα αποτελέσματα της αρχικής υλοποίησης της εφαρμογής. Οι συμβολισμοί είναι οι εξής:

- Run Time - Ο συνολικός χρόνος εκτέλεσης του προγράμματος
- Index Building - Ο χρόνος για την κατασκευή του index
- Queries Time - Ο χρόνος για την εύρεση των εγγύτερων γειτόνων για όλα τα queries
- Queries / Second - Ο αριθμός των queries ανά δευτερόλεπτο, για τα οποία έχουμε βρει τους γείτονες
- Avg Recall - Το μέσο recall όλων των queries
- F1 - Εκτέλεση filtered Vamana για filtered queries
- F0 - Εκτέλεση filtered Vamana για unfiltered queries
- S0 - Εκτέλεση stitched Vamana για unfiltered queries
- Vamana - Εκτέλεση Vamana (Part 1)

	Run Time	Index Building	Queries Time	Queries / second	Avg Recall
F1	40.533s	32s	7s	1428.57	0.904352
F0	164.297s	34s	129s	77.5194	0.78384

S0	56.765s	49s	7s	1428.57	0.925359
Vamana (Part 1)	55.579s	55s	0.303532	32945.455	0.9991

Για κάθε μια από τις παρακάτω βελτιστοποιήσεις, γίνεται σύγκριση με βάση τους παραπάνω χρόνους και αποτελέσματα των αρχικών υλοποιήσεων.

Τα παραπάνω αποτελέσματα έχουν προέρχονται από εκτέλεση με τις εξής παραμέτρους:

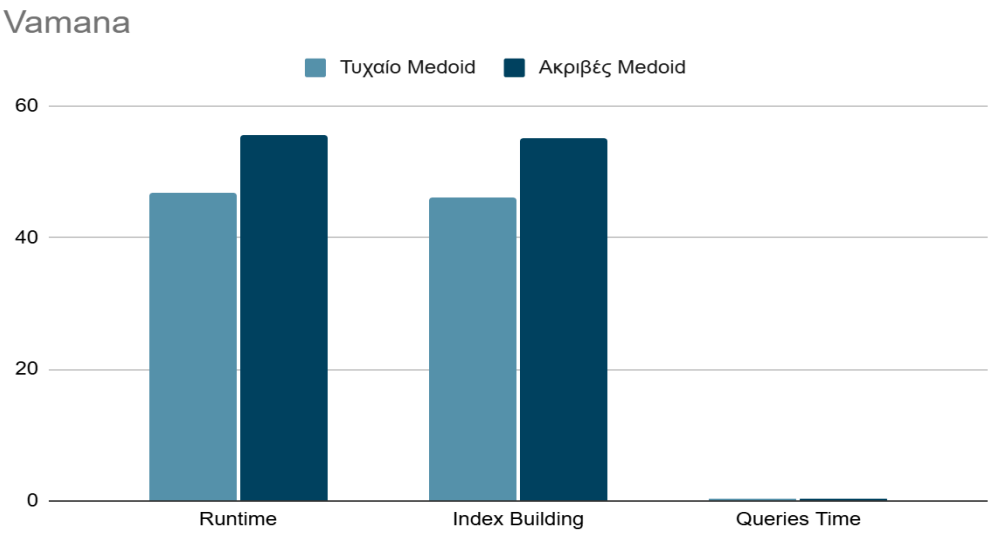
- Για F1 και F0: R = 12, L = 200
- Για S0 και Vamana (Part 1): R = 25, L = 150

*Δεν έχουμε δεδομένα για filtered queries με stitched Vamana καθώς αντιμετωπίσαμε προβλήματα κατά την υλοποίηση του αλγορίθμου τα οποία δεν μπόρεσαν να λυθούν. Αντ' αυτού προσθέσαμε δεδομένα για τον απλό vamana τον οποίο χρησιμοποιεί ο Stitch.

Εύρεση τυχαίου medoid για το vamana index

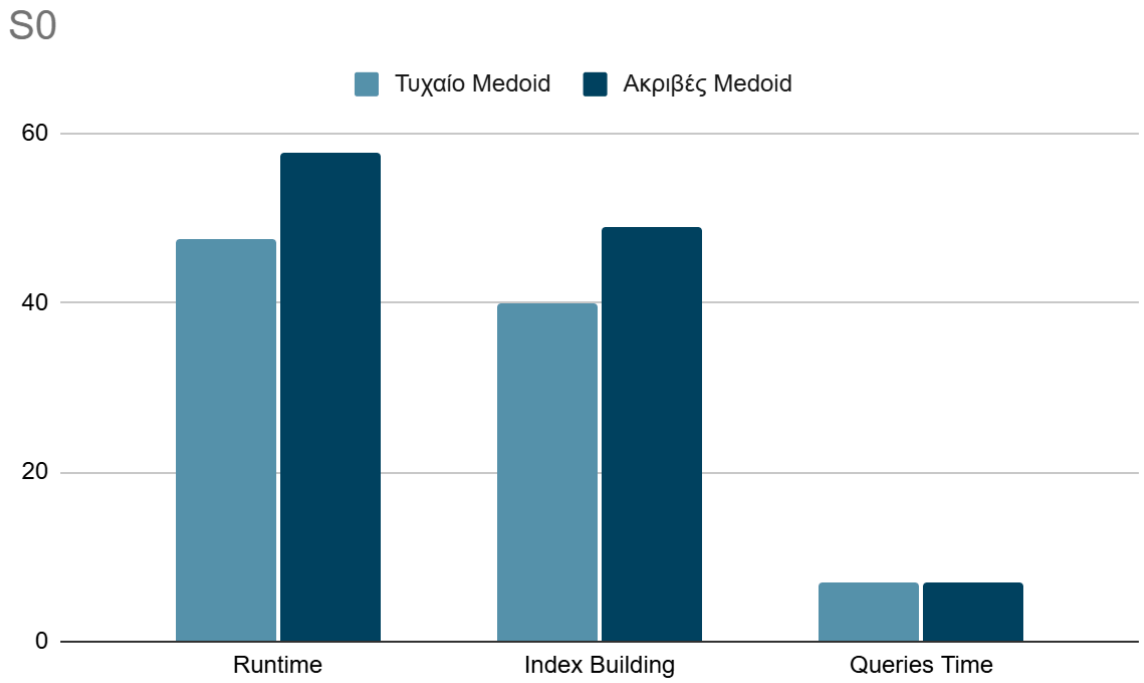
Στην αρχική υλοποίηση υπολογίστηκε το ακριβές medoid, γεγονός που αυξάνει σημαντικά τον χρόνο εκτέλεσης , καθώς πρέπει να βρεθεί η απόσταση κάθε σημείου από το υπόλοιπο dataset. Έγινε επιλογή τυχαίου medoid και παρατηρήθηκαν τα εξής αποτελέσματα:

- Μείωση χρόνου index building αλλά και runtime
- Σταθερότητα σε recall, queries time και queries/second



Όπως παρατηρούμε και στο διάγραμμα, για τον Vamana (Part1), το run time αλλά και index building έχουν μειωθεί κατά 16% (από 55 sec. σε 46 sec.) με την χρήση του τυχαίου medoid.

Αμελητέες είναι οι διαφορές σε **recall (από 0.9993 σε 0.9990)**, queries time (από 0.303532 σε 0.310340) και **queries / microsecond (από 332.387 σε 322.227)**.

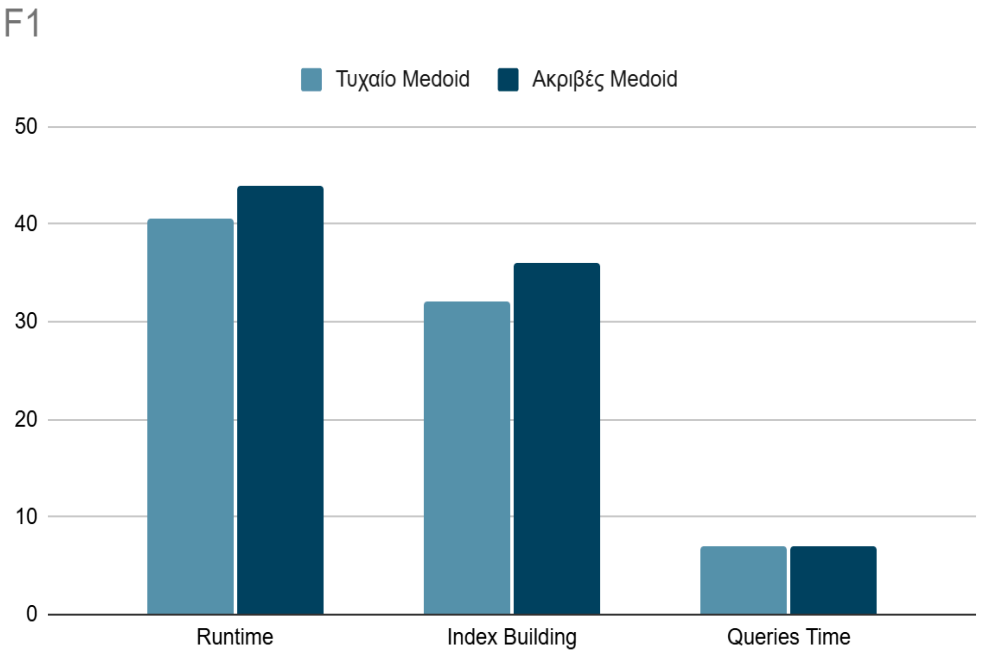


Στο παραπάνω διάγραμμα διακρίνουμε, για το S0, μείωση του run time κατά 17% (από 57 σε 47 sec.) και μείωση του index building κατά 18% (από 49 σε 40 sec), ενώ δεν σημειώθηκε μεταβολή σε queries time (7 sec) και **queries/sec. (1428.57)**. Αμελητέα φαίνεται η μεταβολή στο **recall (από 0.925545 σε 0.924914)**.

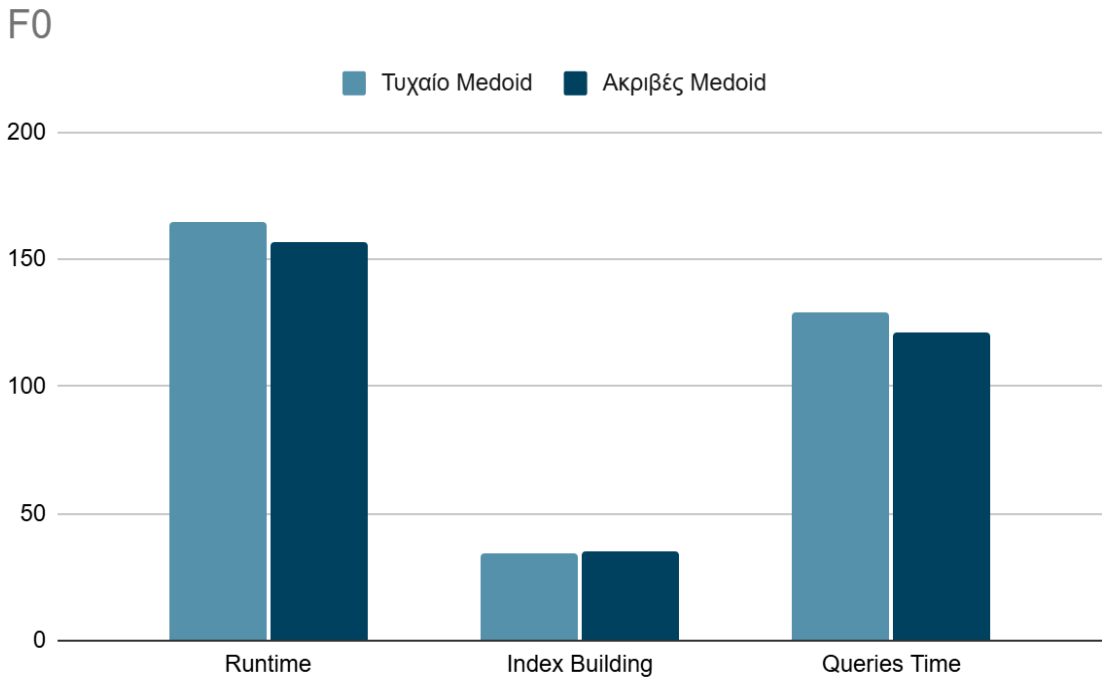
Εύρεση ακριβούς medoid για τον filtered vamana index

Για την δημιουργία του filtered vamana index, το medoid για κάθε φίλτρο επιλέγεται τυχαία, γεγονός που δημιουργεί αμφιβολίες για την ακρίβεια των αποτελεσμάτων. Έγινε δοκιμή υπολογισμού του ακριβούς medoid για κάθε φίλτρο και παρατηρήθηκαν τα εξής:

- Για F1, μικρή αύξηση σε run time και index building , αλλά σταθερότητα σε recall, queries time και queries/second
- Για F0, σταθερότητα σε recall και index building, μικρή μείωση σε run time και queries time και αύξηση σε queries/second



Στο διάγραμμα του F1 διακρίνεται η μικρή αύξηση του run time κατά 8% (από 40.533 σε 43.994 sec) και index building κατά 12.5% (από 32 σε 36 sec), αλλά και η σταθερότητα του queries time (7 sec). Σταθερά μένουν επίσης τα **queries / sec (1428.57)**, ενώ αμελητέα είναι η διαφορά στο **recall (από 0.904352 σε 0.907389)**.



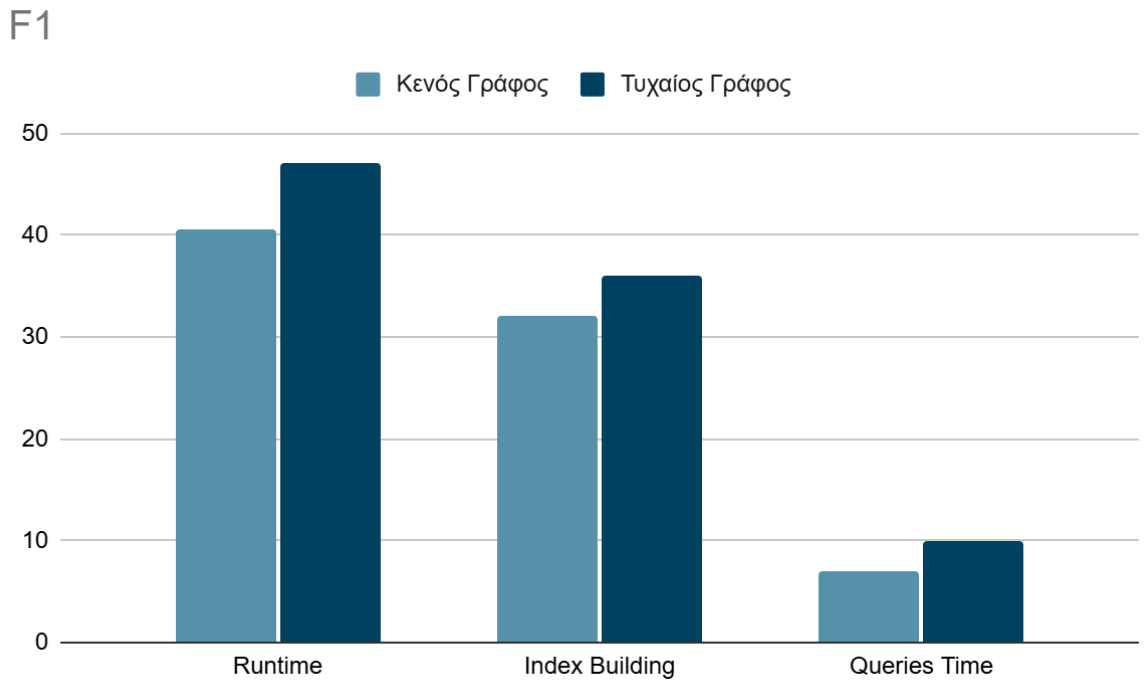
Στο διάγραμμα του F0 φαίνεται η κατά 4.5% μείωση του run time (από 164.297 σε 157 sec), η κατά 6.2% μείωση του queries time (από 129 σε 121 sec) και η ενός δευτερολέπτου,

αμελητέα αύξηση του index building (από 34 σε 35 sec). Κατά **6.5% αυξήθηκαν τα queries / second (από 77.5194 σε 82.6446)**, ενώ ανεπαίσθητη είναι η αύξηση του **recall (από 0.7838 σε 0.7974)**.

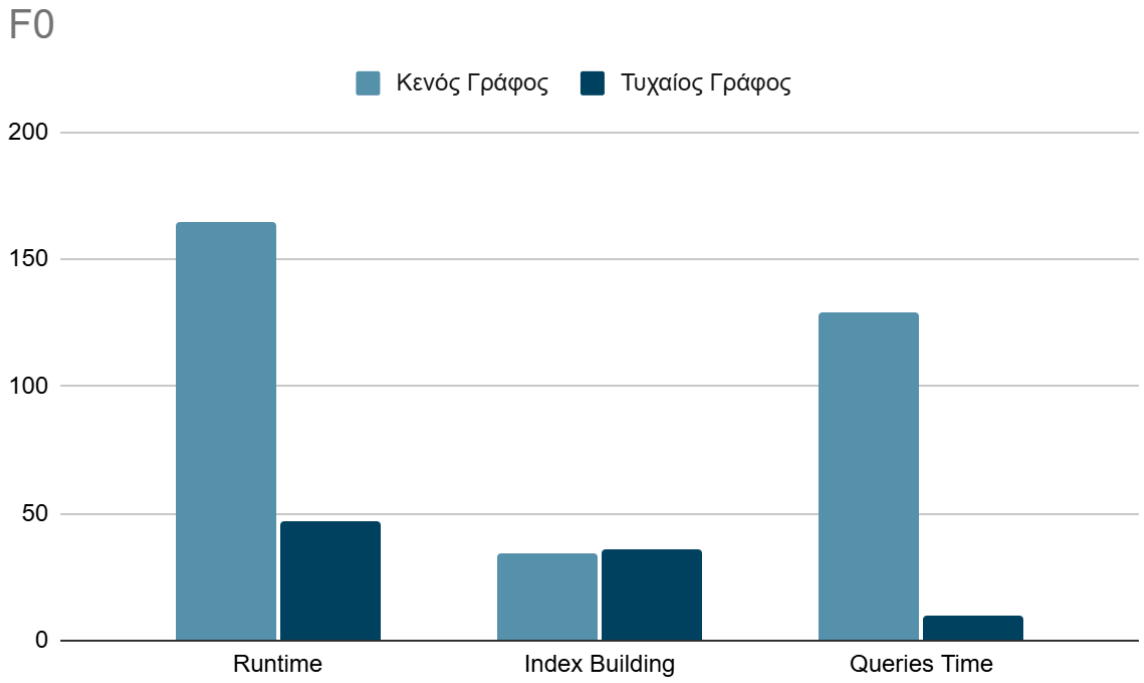
Αρχικοποίηση γράφου με τυχαίες ακμές

Κατά την δημιουργία του filtered vamana index, ο γράφος είναι στην αρχή κενός και στην συνέχεια προστίθενται ακμές μεταξύ σημείων ίδιου φίλτρου. Εδώ ωστόσο, έγινε αρχικοποίηση του γράφου με τυχαίες ακμές ανεξαρτήτως φίλτρου, με σκοπό να φανεί ενδεχόμενη αλλαγή στο αποτέλεσμα για τα unfiltered queries. Παρατηρήθηκαν τα παρακάτω:

- Για F1, αύξηση σε run time , index building και queries time, μείωση στα queries / second και εκπληκτική μείωση στο recall.
- Για F0, μεγάλη μείωση σε run time , queries time αλλά και σε queries / second και recall (αντίθετα με το αναμενόμενο αποτέλεσμα), ενώ ελάχιστη είναι η αύξηση στο index building.



Στο διάγραμμα του F1 φαίνεται η κατά 16% αύξηση του run time (από 40.533 σε 47 sec), η κατά 12.5% αύξηση του index building (από 32 σε 36 sec) και η κατά 42.9% αύξηση του queries time (από 7 σε 10 sec). Κατά **30% μειώθηκαν τα queries / second (από 1428.57 σε 1000)** και κατά **78% (από 0.904352 σε 0.19893) το recall.**

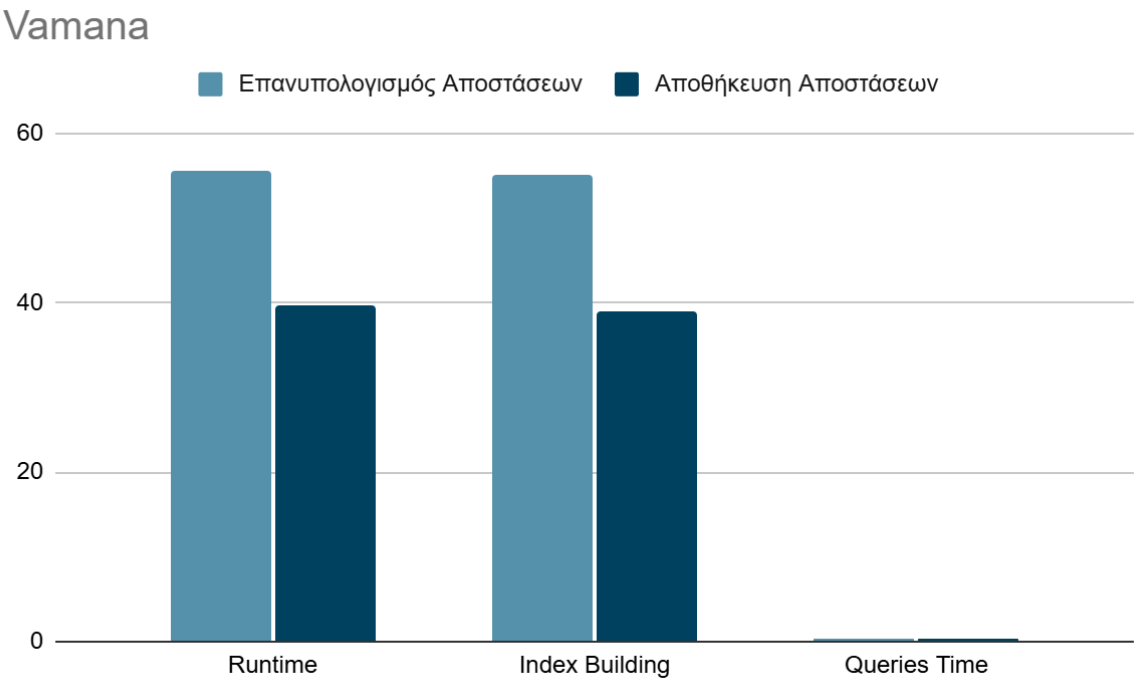


Στο διάγραμμα του F0 παρατηρείται η μεγάλη μείωση του run time κατά 71.2% (από 164.297 σε 47.206 sec), του queries time κατά 92.3% (από 129 σε 10 sec) και η αύξηση του index building κατά 5.9% (από 34 σε 36 sec). Εκπληκτική είναι η **αύξηση σε queries / second κατά 1190% (από 77.5194 σε 1000)** αλλά και η **μείωση στο recall κατά 63% (από 0.78384 σε 0.289964)**.

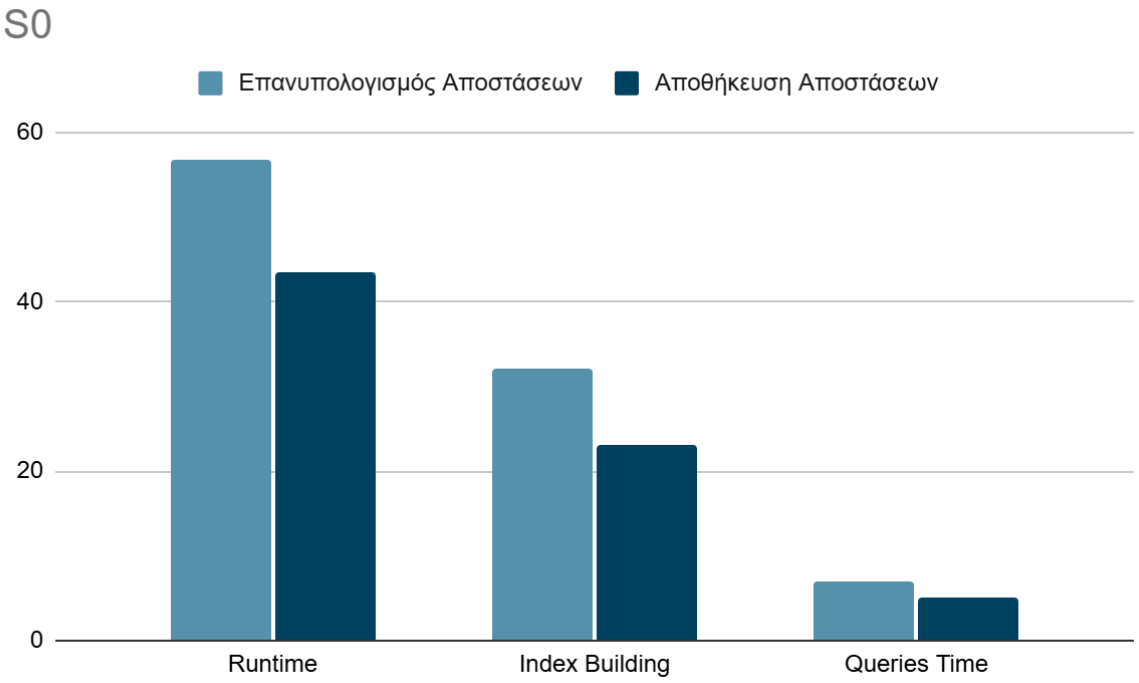
Αποθήκευση αποστάσεων σε 1-d matrix για την αποφυγή επανυπολογισμού

Για την αποφυγή επανυπολογισμών αποστάσεων από κάποιο σημείο σε άλλο, που έχουν ήδη βρεθεί στην ίδια κλήση της συνάρτησης, κάθε απόσταση που υπολογίζεται για πρώτη φορά, αποθηκεύεται σε ένα 1-d matrix το οποίο είναι ένα vector, ώστε να μπορεί να ανακτηθεί. Παρατηρήθηκαν:

- Για Vamana (Part 1), γενική μείωση σε όλους τους παράγοντες, με σταθερότητα στο recall
- Για S0, μείωση σε run time, index building, queries time, αύξηση σε queries / second, σταθερότητα στο recall
- Για F1, ομοίως με S0
- Για F0, ομοίως με S0, F1

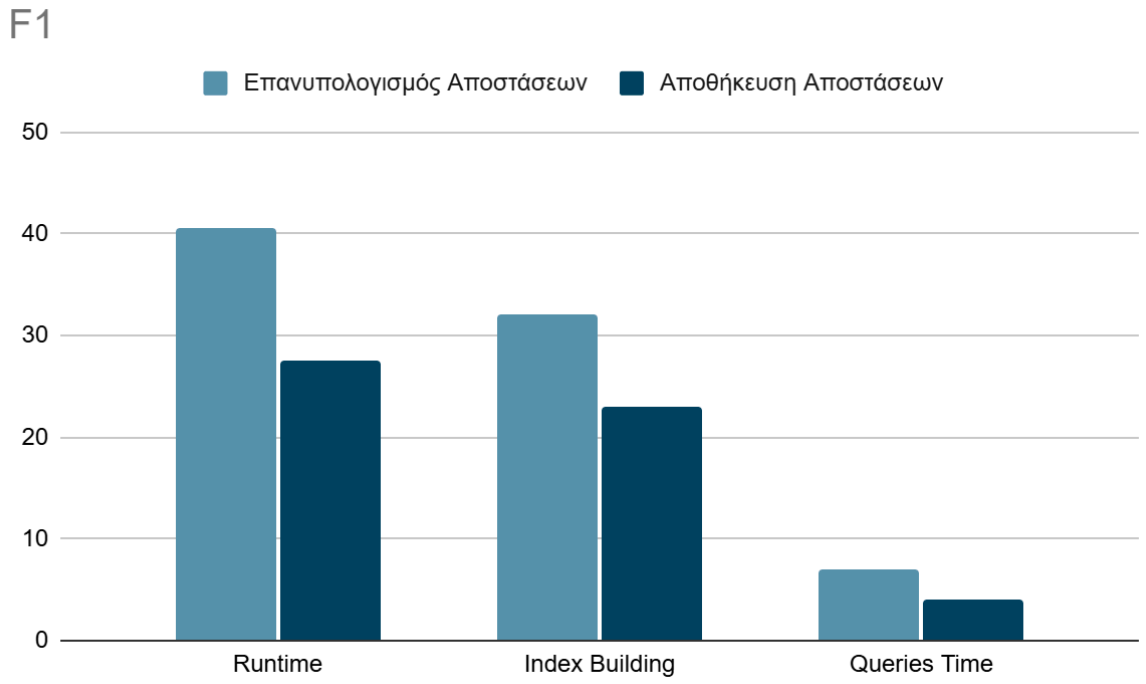


Στο διάγραμμα του Vamana (Part1) διακρίνεται η μείωση του run time κατά 28.5% (από 55.577 σε 39.735 sec), του index building κατά 29% (από 55 σε 39 sec), και του queries time κατά 39.1% (από 0.3035 σε 0.1846 sec). Μεγάλη είναι η μείωση των queries / second, κατά 98% (από 32945.455 σε 541.515), ενώ το recall μένει σταθερό (στο 0.999).

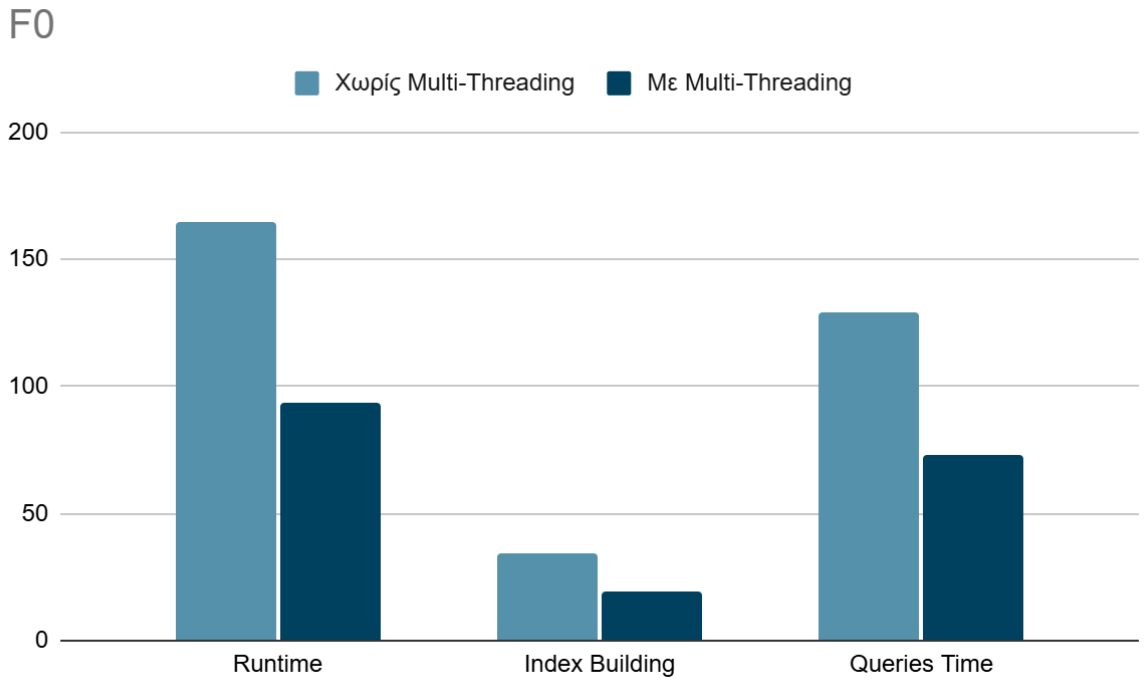


Παρατηρείται στο διάγραμμα του S0, η μείωση του run time κατά 23.4% (από 56.765 σε 43.462 sec), του index building κατά 28.5% (από 49 σε 35 sec) και του queries time κατά το

ίδιο ποσοστό (από 7 σε 5 sec). Η **αύξηση των queries / second είναι της τάξης του 40% (από 1428.57 σε 2000)**, ενώ το **recall παραμένει σταθερό (στο 0.925)**.



Φαίνεται στο διάγραμμα του F1 η μείωση του run time κατά 32% (από 40.533 σε 27.554 sec), του index building κατά 28.1% (από 32 σε 23 sec) και του queries time κατά 42.8% (από 7 σε 4 sec). Έχουμε επίσης **αύξηση των queries / second κατά 75% (από 1428.57 σε 2500)** και **ελάχιστη αύξηση στο recall (από 0.904 σε 0.906)**.

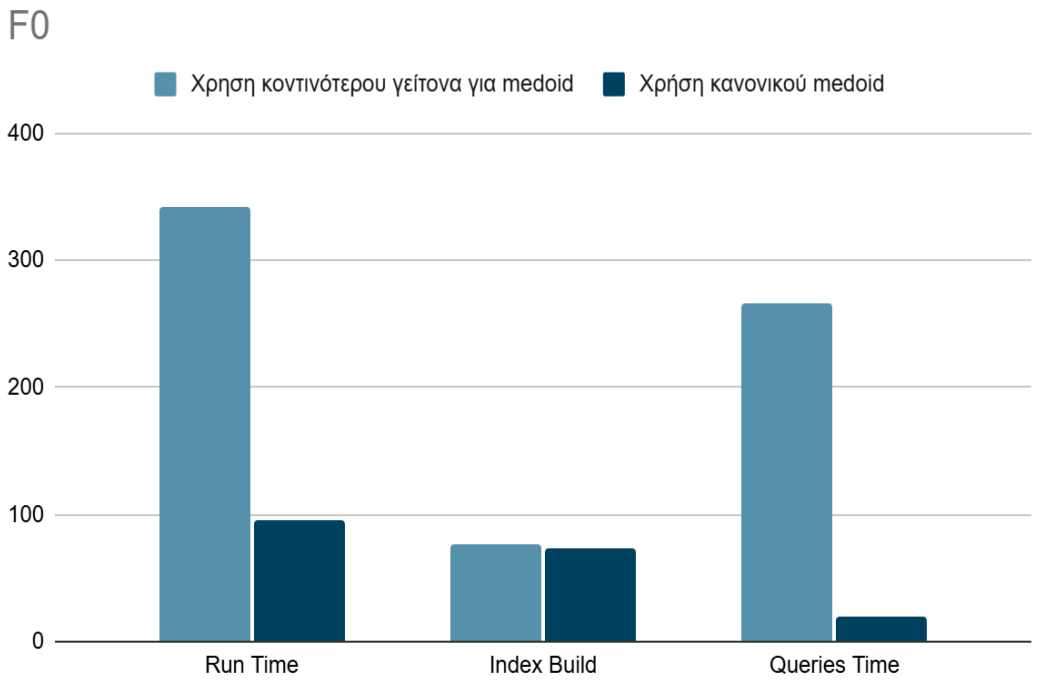


Τέλος, στο διάγραμμα του F0, διακρίνεται η μείωση του run time κατά 42.1% (από 164.297 σε 95 sec), του index building κατά 35.2% (από 34 σε 22 sec) και του queries time κατά 44.1% (από 129 σε 72 sec). Τα **queries / second έχουν αυξηθεί κατά 79% (από 77.519 σε 138.889)** ενώ το **recall έχει υποστεί ανεπαίσθητη μείωση (από 0.7838 σε 0.7775)**.

Χρήση κοντινότερων γειτόνων κάθε φίλτρου ως medoid για unfiltered queries

Τα unfiltered queries στον filtered vamana graph από το προηγούμενο κιόλας παραδοτέο είχαμε παρατηρήσει οτι παρουσιάσουν πολύ και ακρίβεια αποτελέσματος (recall), γεγονός που δικαιολογείται από την υλοποίηση του αλγορίθμους ο οποίος επιλέγει ακμές με κοινά φίλτρα και έτσι αποκλείει ακμές διαφορετικών φίλτρων που ενδέχεται να έχουν μικρότερη μεταξύ τους απόσταση. Έτσι, βασισμένοι στην ανάλυση του συναδέλφου, υπολογίσαμε πρώτα τον ένα κοντινότερο γείτονα κάθε medoid και χρησιμοποιήσαμε αυτό το αποτέλεσμα ως medoid. Επίσης διπλασιάσαμε το L ώστε να εξετάζονται περισσότεροι κόμβοι. Τα αποτελέσματα ήταν τα εξής:

- Δραματική αύξηση του recall.
- Σημαντική αύξηση του run time

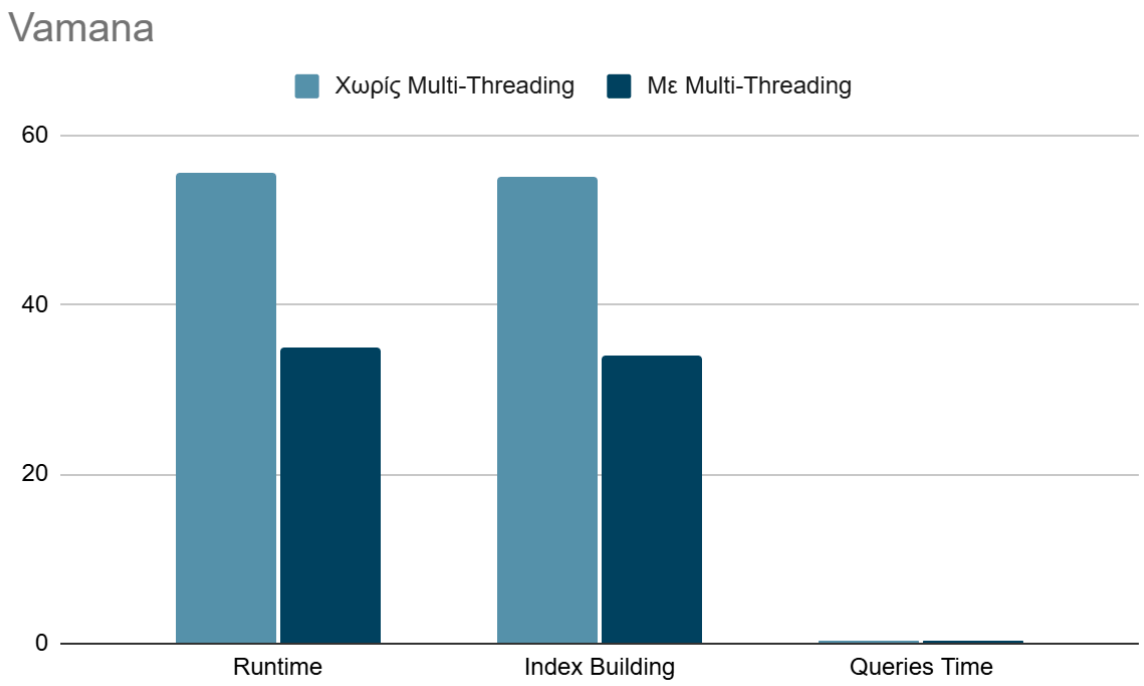


Στο παραπάνω διάγραμμα για τον F0 φαίνεται η αύξηση του run time κατά 72.28% (από 94.981 σε 342.739sec), του queries time κατά 92.48% (από 20 σε 266 sec) ενω το index build δεν επηρεάζεται. Τα **queries / second έχουν μειωθεί κατά 98.4% (από 32945.455 σε 505.477)** ενώ το **recall παραμένει σταθερό (στο 0.999)**.

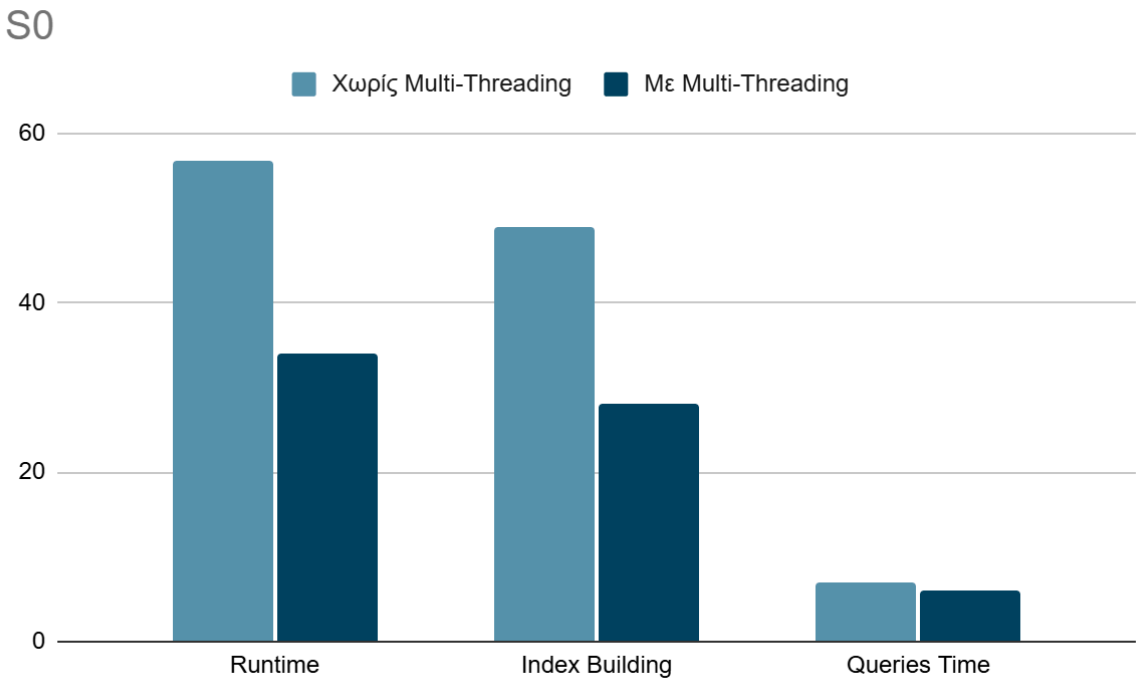
Χρήση multi-threading

Ως τελευταία βελτίωση, προστέθηκε multi-threading στην τελική μορφή της εφαρμογής, προκειμένου να παραλληλοποιηθούν όσες διαδικασίες είναι εφικτό, ώστε να βελτιωθεί ακόμη περισσότερο η απόδοση της εφαρμογής. Τα αποτελέσματα ήταν:

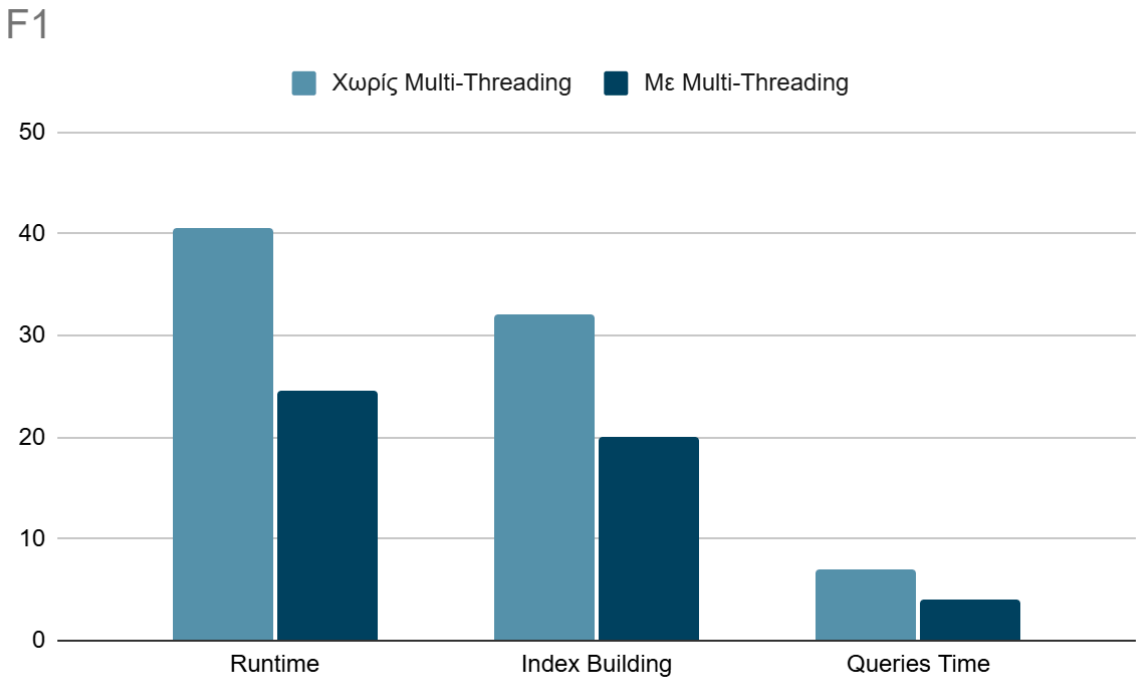
- Για Vamana (Part1), μείωση σε όλους τους παράγοντες, σταθερό recall
- Για S0, μείωση σε run time, index building και queries time, αύξηση των queries / second, σταθερό recall
- Για F1, ομοίως με S0
- Για F0, ομοίως με S0 και F1



Στο παραπάνω διάγραμμα για τον Vamana (Part 1) φαίνεται η μείωση του run time κατά 37% (από 55.579 σε 35 sec), του index building κατά 38.2% (από 55 σε 34 sec) και του queries time κατά 34.8% (από 0.3035 σε 0.1978 sec). Τα **queries / second έχουν μειωθεί κατά 98.4% (από 32945.455 σε 505.477)** ενώ το **recall παραμένει σταθερό (στο 0.999)**.

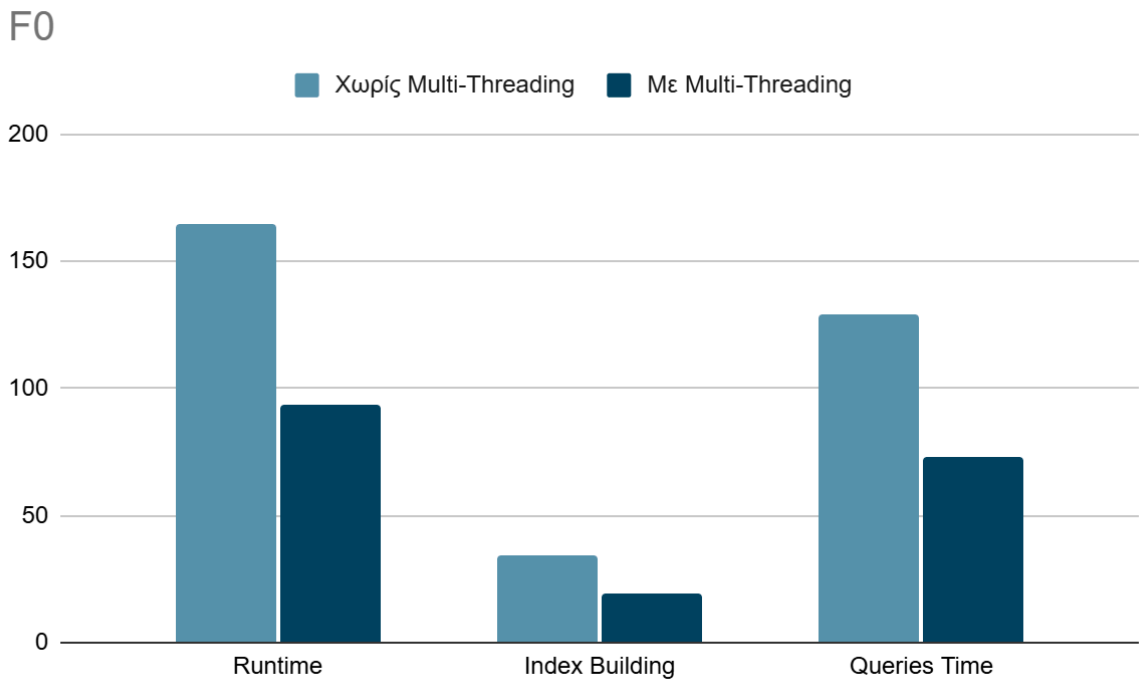


Στο διάγραμμα του S0 διακρίνεται η μείωση του run time, σε ποσοστό 40.1% (από 56.765 σε 34 sec), του index building, σε ποσοστό 42.8% (από 49 σε 28 sec) και του queries time, σε ποσοστό 14.3% (από 7 σε 6 sec). **Τα queries / second αυξάνονται κατά 16.6% (από 1428.57 σε 1666.67)**, ενώ το **recall παραμένει σταθερό (στο 0.9253)**.



Το παραπάνω διάγραμμα για F1 φανερώνει την μείωση του run time κατά 39.3% (από 40.533 σε 24.6 sec), του index building κατά 37.5% (από 32 σε 20 sec) και του queries time κατά

42.8% (από 7 σε 4 sec). Παρατηρείται επίσης **αύξηση των queries / second κατά 75% (από 1428.57 σε 2500)** και μια **μιδαμινή αύξηση του recall (από 0.9043 σε 9064).**



Τέλος, στο διάγραμμα για F0 παρατηρείται μείωση στο run time κατά 43% (από 164.297 σε 93.6 sec), στο index building κατά 44.1% (από 34 σε 19 sec) και στο queries time κατά 43.4% (από 129 σε 73 sec). **Τα queries / second έχουν αυξηθεί κατά 76.6% (από 77.519 σε 136.6), ενώ ελάχιστα φαίνεται να έχει μειωθεί το recall (από 0.7838 σε 0.7788).**

Εκτελέσεις με διαφορετικές παραμέτρους

Με βάση όλες τις παραπάνω βελτιστοποιήσεις και μετρήσεις, καταλήξαμε στην τελική εκδοχή της εφαρμογής, η οποία θεωρούμε ότι δίνει τον βέλτιστο συνδυασμό χρόνου εκτέλεσης και ακρίβειας αποτελεσμάτων. Δοκιμάσαμε λοιπόν αυτή την εκδοχή με διαφορετικές παραμέτρους και ενδεικτίκα παρουσιάζουμε κάποια αποτελέσματα:

Για R = 12, L = 150

	Run Time	Index Building	Queries Time	Queries / second	Avg Recall
F1	18s	13s	4s	2500	0.90
F0	65s	14s	50s	200	0.74
S0	56.765s	49s	7s	1428.57	0.87

Για R = 25, L = 200

	Run Time	Index Building	Queries Time	Queries / second	Avg Recall
F1	33s	27s	7s	1428.57	0.87
F0	118s	27s	90s	111.11	0.73
S0	56.765s	49s	7s	1428.57	0.925359

Λαμβάνοντας υπόψη τις αρχικές μετρήσεις, αλλά και τις δοκιμαστικές εκτελέσεις με διαφορετικές παραμέτρους, συμπεραίνουμε ότι σε ένα εύλογο εύρος τιμών για το L, (π.χ 150 - 250), δεν επηρεάζεται το recall, ωστόσο η μεταβολή του χρόνου εκτέλεσης είναι ανάλογη με αυτή του L. Επιπλέον, ανάλογη είναι και η μεταβολή του χρόνου εκτέλεσης ως προς του R, με το recall όμως να μεταβάλλεται αντίστροφα.

Συμπεράσματα

Καταληκτικά λοιπόν, μελετώντας προσεκτικά τα αποτελέσματα της κάθε μιας από τις παραπάνω βελτιστοποιήσεις, αλλά και τις δοκιμαστικές εκτελέσεις, φτάσαμε στην τελική εκδοχή της εφαρμογής η υλοποίηση της οποίας συνδυάζει τις πιο αποδοτικές από τις παραπάνω βελτιώσεις.

Συγκεκριμένα, το medoid υπολογίζεται τυχαία, καθώς η επίπτωση στο recall είναι αμελητέα, ενώ μειώνεται σημαντικά ο χρόνος εκτέλεσης. Η απόσταση ενός σημείου από κάποιο άλλο υπολογίζεται και αποθηκεύεται σε 1-d matrix για να αποφευχθεί ο περιττός επανυπολογισμός και ταξινομείται με την χρήση priority queue. Για unfiltered queries χρησιμοποιείται ως medoid ένας κοντινός γείτονας, γεγονός που βελτιώνει σημαντικά το recall, ενώ όπου κρίνεται απαραίτητο εφαρμόζεται multi-threading για την παραλληλοποίηση εργασιών, άρα και ταχύτερη εκτέλεση του προγράμματος.

Τέλος, οι παράμετροι που μας δίνουν τον βέλτιστο δυνατό συνδυασμό recall και χρόνου εκτέλεσης είναι τιμές του L στο εύρος [150 - 300] και τιμές του R στο εύρος [10 - 25].